

CanPlan

Realization Document

Wout Jansegers
Student Bachelor Applied Computer Science

Table of Contents

1. INTRODUCTION	3
2. ANALYSIS	4
3. REALIZATION CANPLAN	6
3.1. Existing Situation and Project Goals	6
3.2. Rebranding and Redesign of the Application	7
3.3. Research Into Cloud Expansion	9
3.4. Admin Dashboard Design	11
3.5. Implementation of New Features	13
3.6. User Feedback and Surveys	14
3.7. Challenges During Development	14
4. CONCLUSION	15
REFERENCE LIST	16

1. Introduction

The internship took place at CanAssist, an organization based in Victoria, British Columbia, that focuses on improving accessibility and inclusion through technological solutions for people, particularly individuals with disabilities.

This document describes the realization of my internship project. During this internship, I worked on improving and expanding the existing CanPlan application through redesign, accessibility improvements, feature implementation, and research into future cloud-based expansion possibilities. The original application functioned completely offline and was designed to support users in completing tasks through step-by-step guidance.

The purpose of this document is to explain the technical implementation of this extension. The focus is on the decisions that were made, the technologies that were selected, and how the final solution was developed.

This document builds on the previously created project plan, in which the problem statement, objectives, and general approach were defined. While the project plan focused on what needed to be achieved, this realization document explains how the solution was implemented in practice.

The document is structured as follows. First, the analysis is discussed, where different technical options were explored. Next, the realization of the system is described. Finally, a conclusion is provided, including a reflection on the project and possible future improvements.

2. Analysis

This chapter describes the preparation phase of the project and the different technical options that were investigated before starting the implementation. The goal of this analysis was to determine which technologies and tools would best support the transition from a local application to a cloud-based platform.

The existing Flutter application worked entirely offline. To enable new features such as authentication, synchronization across devices, and an admin dashboard, it was necessary to introduce a backend system. Several backend solutions were explored, including Firebase, Supabase, and a custom backend.

Firebase was considered because of its fast setup and strong integration with Flutter. It offers built-in authentication and real-time synchronization, which makes it attractive for rapid development. However, Firebase uses a NoSQL database, which is less suitable for applications with complex relationships between users, roles, and tasks. In addition, there is a dependency on the platform, which can limit flexibility in the long term.

Supabase was also evaluated as an alternative. It provides a relational database based on PostgreSQL and includes built-in authentication and security features such as row-level security. This makes it more suitable for structured data and role-based access control. While Supabase offers more flexibility than Firebase, it requires more technical knowledge and has a smaller ecosystem.

A third option was to develop a custom backend using technologies such as Node.js or Spring Boot. This approach requires more development effort, but it provides full control over the system. It allows complex business logic to be implemented in a structured way and avoids limitations imposed by external platforms.

When comparing these options, several criteria were taken into account, including Flutter integration, roles and permissions, dashboard support, development speed, scalability, and technical complexity. Additional emphasis was placed on scalability, Flutter integration, and dashboard support, as these are important requirements for the future development of the application. The results of this comparison are shown in the Weighted Decision Matrix. While Firebase scored highly on ease of use and development speed, it scored lower on flexibility and support for complex data structures. Supabase provided a strong balance between functionality and development effort. The custom backend required more effort to implement but achieved the highest overall score due to its flexibility, control, and long-term scalability.

Based on this comparison, a custom backend using Spring Boot was selected. This choice was motivated by the need for a structured architecture, strong security features, and the ability to implement complex logic. In addition, it aligns well with enterprise development practices and provides strong academic value.

In terms of data storage, PostgreSQL was chosen as the database. The application requires handling relationships between users, roles, and tasks, which makes a relational database more appropriate. PostgreSQL also offers strong query capabilities and ensures data consistency.

For the frontend, the existing Flutter application was maintained. This avoided unnecessary redevelopment and allowed reuse of the existing codebase. For the admin dashboard, a separate web-based solution was required. Vue was selected because it is efficient for building structured dashboards and supports rapid development of user interfaces focused on data management.

Finally, different hosting solutions were explored, including Vercel, AWS, and DigitalOcean. While platforms such as Vercel offer simplicity, they are less suitable for custom backend systems. AWS provides extensive features but introduces complexity. A VPS-based solution was chosen as it offers a good balance between control, cost, and flexibility.

In conclusion, the analysis phase provided a clear overview of the available options and their trade-offs. The final technology stack was selected based on a balance between flexibility, scalability, and long-term maintainability, rather than purely on ease of implementation.

CanAssist Research		Options					
		Firebase		Supabase		Eigen Backend	
Criteria	Weight	Score	Total	Score	Total	Score	Total
Flutter integration	4	5	20	5	20	4	16
Roles and Permissions	5	2	10	4	20	5	25
Dashboard support	4	3	12	4	16	5	20
Development speed	2	5	10	5	10	2	4
Scalability	5	3	15	4	20	5	25
Technical complexity	1	3	3	3	3	1	1
-	Total	70		89		91	

Weighted Decision Matrix

3. Realization CanPlan

3.1. Existing Situation and Project Goals

At the start of the internship, CanPlan already existed as a functional mobile application developed for people who benefit from step-by-step guidance during daily activities. The application mainly focused on local storage and offline functionality, allowing users to create and complete tasks directly on their device. Although the existing version already provided useful functionality, several areas could still be improved both visually and technically.

The original application contained an outdated interface design, limited scalability for future expansion, and several usability challenges related to accessibility and navigation clarity. In addition, the application architecture was mainly designed for local usage, which limited future possibilities such as cloud synchronization, remote task management, and multi-user support.

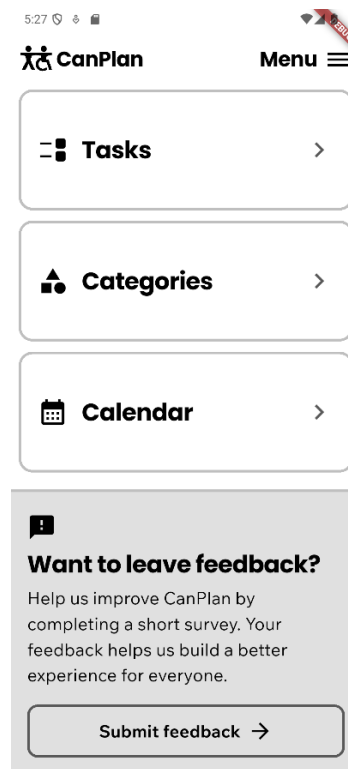
The main goal of this internship was therefore not only to improve the existing application visually, but also to explore how CanPlan could evolve into a more scalable and future-oriented platform.

The project focused on several important objectives:

- Redesigning and rebranding the entire application
- Improving accessibility and usability
- Implementing new features and workflows
- Researching cloud-based expansion possibilities
- Exploring backend and database solutions
- Designing a future admin dashboard
- Improving scalability and maintainability for future development



Original Mobile Dashboard



New Mobile Dashboard

3.2. Rebranding and Redesign of the Application

One of the largest parts of the internship involved the complete redesign and rebranding of the CanPlan application. The goal of this redesign was to modernize the application while simultaneously improving accessibility, consistency, and overall usability for the target audience.

The previous version of the application contained several inconsistencies in layout, colors, spacing, and navigation structure. During the redesign process, the application was reviewed screen by screen to identify areas that could be improved both visually and functionally.

The redesign process focused heavily on accessibility and cognitive simplicity. Since many users of CanPlan benefit from clear visual guidance and minimal distractions, the interface was redesigned with a cleaner and more structured layout.

Several important improvements were implemented:

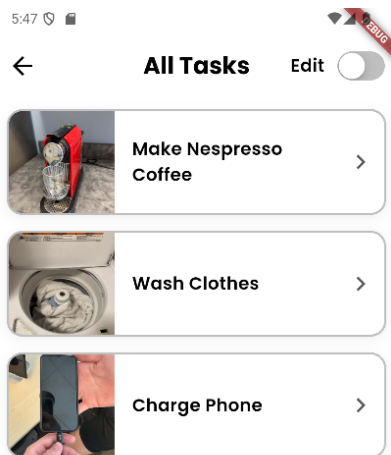
- Improved typography and readability
- More consistent spacing and layouts
- Simplified navigation structure
- Better visual hierarchy
- Clearer buttons and interaction elements
- Reduced visual clutter
- Improved consistency between screens
- Modernized color palette and branding

The redesign included nearly every major screen within the application, including:

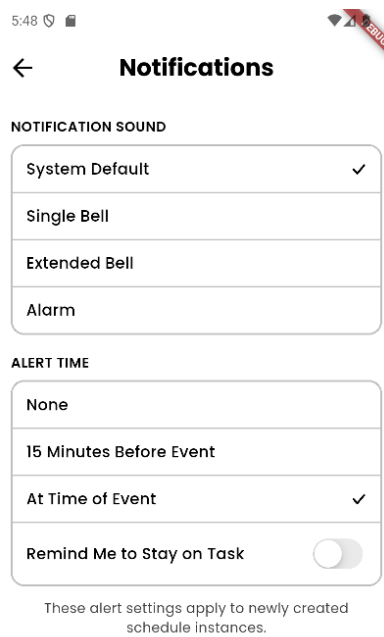
- Dashboard
- Task overview screens
- Task creation flow
- Task execution screens
- Settings pages
- Category management
- Tutorial and information screens

In addition, reusable UI components were created to ensure visual consistency throughout the application. This included reusable buttons, cards, navigation elements, dialogs, and other interface components.

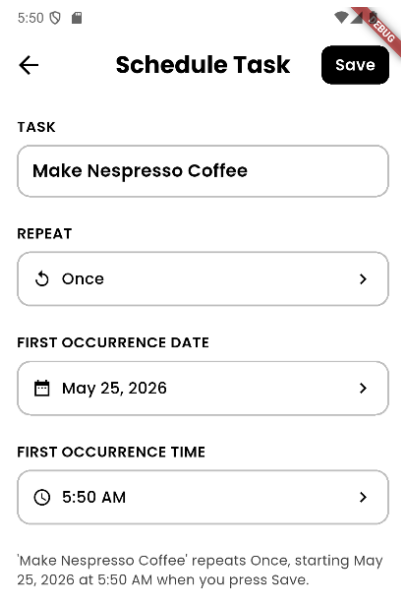
Throughout the redesign process, regular feedback sessions were held together with the App Design Consultant to review accessibility, usability, and consistency within the application.



New All-Tasks Screen



New Notifications Settings



New Schedule Screen

3.3. Research Into Cloud Expansion

Another major part of the internship focused on researching how CanPlan could potentially evolve from a local application into a cloud-supported platform.

The existing version of CanPlan mainly stores data locally on the device. While this approach offers strong offline functionality and simplicity, it also introduces limitations regarding synchronization, remote management, backups, and multi-device usage.

Because of this, extensive research was conducted into possible cloud-based architectures and technologies that could support future expansion.

Several topics were investigated during this research phase.

BACKEND TECHNOLOGIES

Different backend technologies and frameworks were researched and compared based on scalability, maintainability, security, and long-term feasibility.

The researched technologies included:

- Spring Boot
- Firebase
- Node.js
- Supabase

Special attention was given to Spring Boot because of its strong enterprise support, scalability, and structured architecture.

DATABASE RESEARCH

Different database systems were also compared to determine which solution would best fit the future requirements of CanPlan.

The researched database technologies included:

- PostgreSQL
- MySQL

During this research, PostgreSQL appeared to be the strongest candidate because of its advanced relational capabilities, scalability, and strong support for structured data models.

HOSTING AND INFRASTRUCTURE RESEARCH

Several hosting solutions were investigated to evaluate deployment possibilities and future scalability.

The following platforms were researched:

- AWS
- DigitalOcean
- Vercel

The research focused on:

- pricing
- scalability
- deployment complexity
- Canadian data hosting possibilities
- Maintainability

PRIVACY AND FIPPA CONSIDERATIONS

Since CanAssist operates within an educational and public environment in British Columbia, privacy regulations such as FIPPA (Freedom of Information and Protection of Privacy Act) were an important consideration throughout the research process.

The research therefore also focused on:

- Canadian data residency
- secure data storage
- encrypted communication
- authentication security
- role-based access control
- privacy compliance

This research created a strong foundation for future expansion of the application into a scalable cloud-supported platform.

3.4. Admin Dashboard Design

Besides improving the mobile application itself, research and design work was also performed for a future web-based admin dashboard.

The purpose of this dashboard is to provide administrators, supervisors, and caregivers with tools to manage users, tasks, and application data more efficiently.

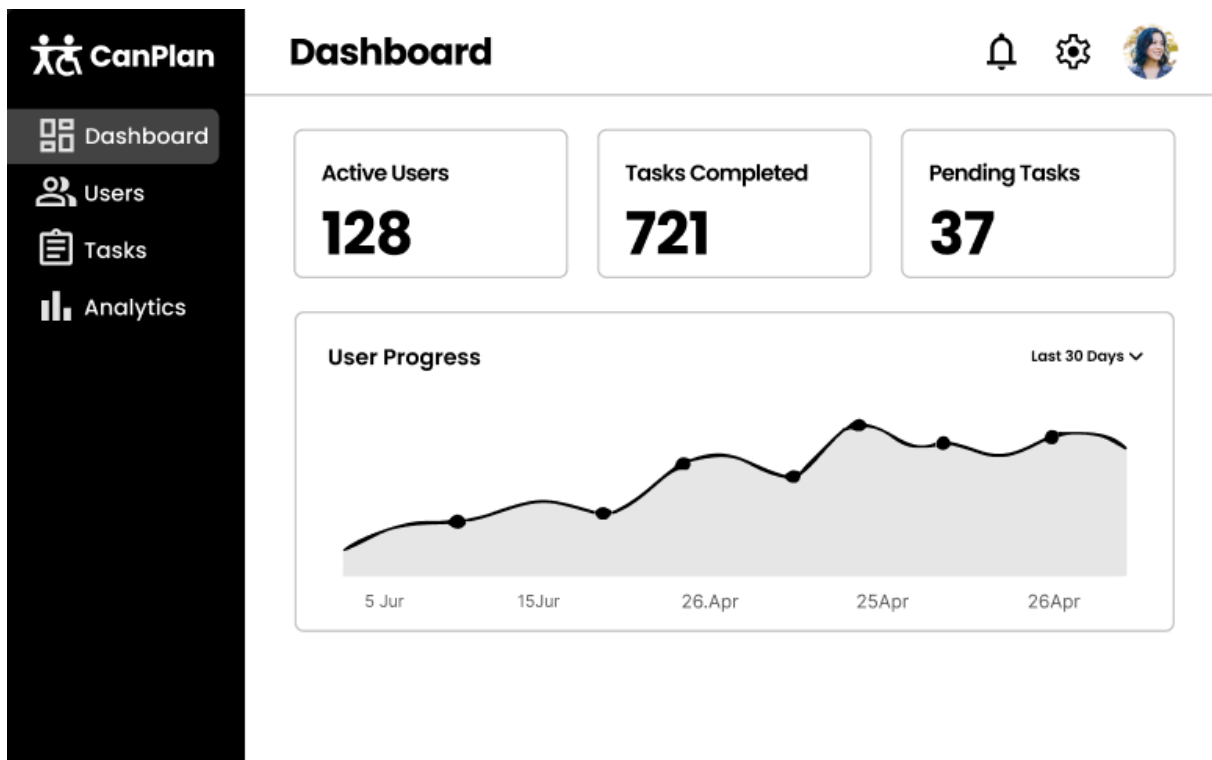
The dashboard was designed in Figma and focused on creating a structured and modular interface that could later be implemented as a separate web application.

Several dashboard sections were designed, including:

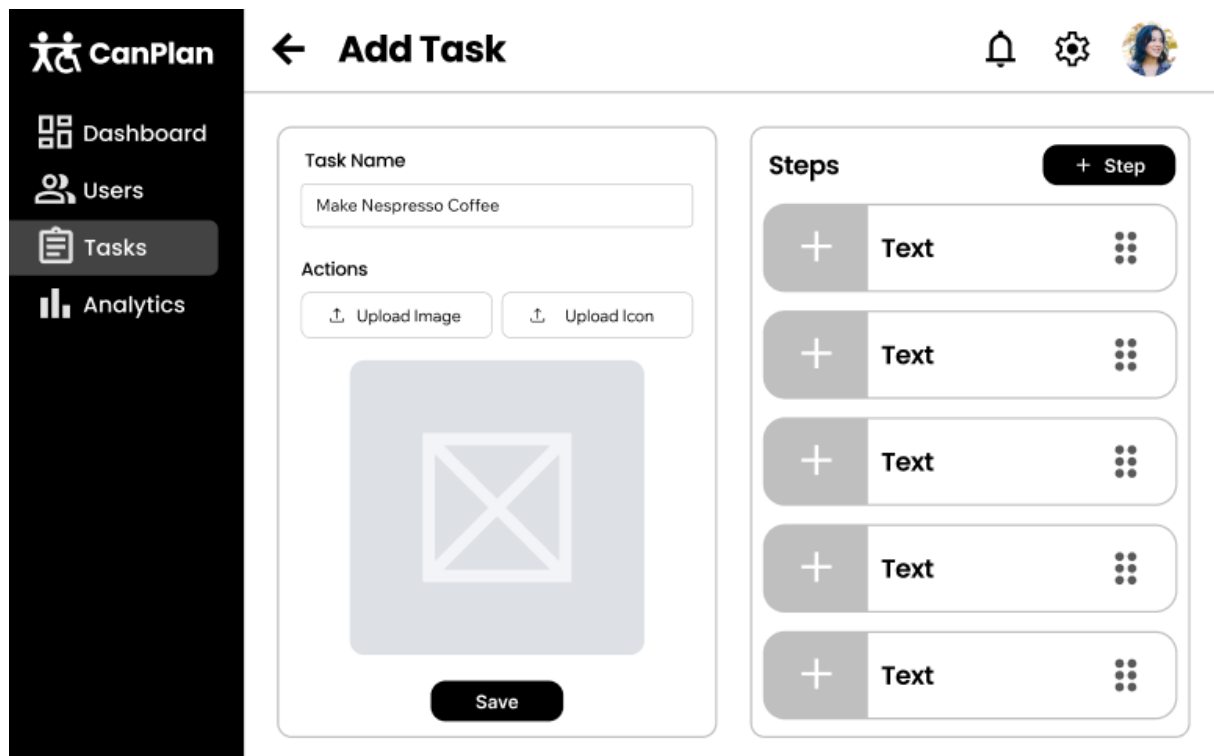
- Dashboard overview
- User management
- Task management
- Statistics and analytics
- Task creation and editing
- Settings and administration

A strong focus was placed on reusable components and modular design. This allows the interface to remain consistent while also simplifying future development and maintenance.

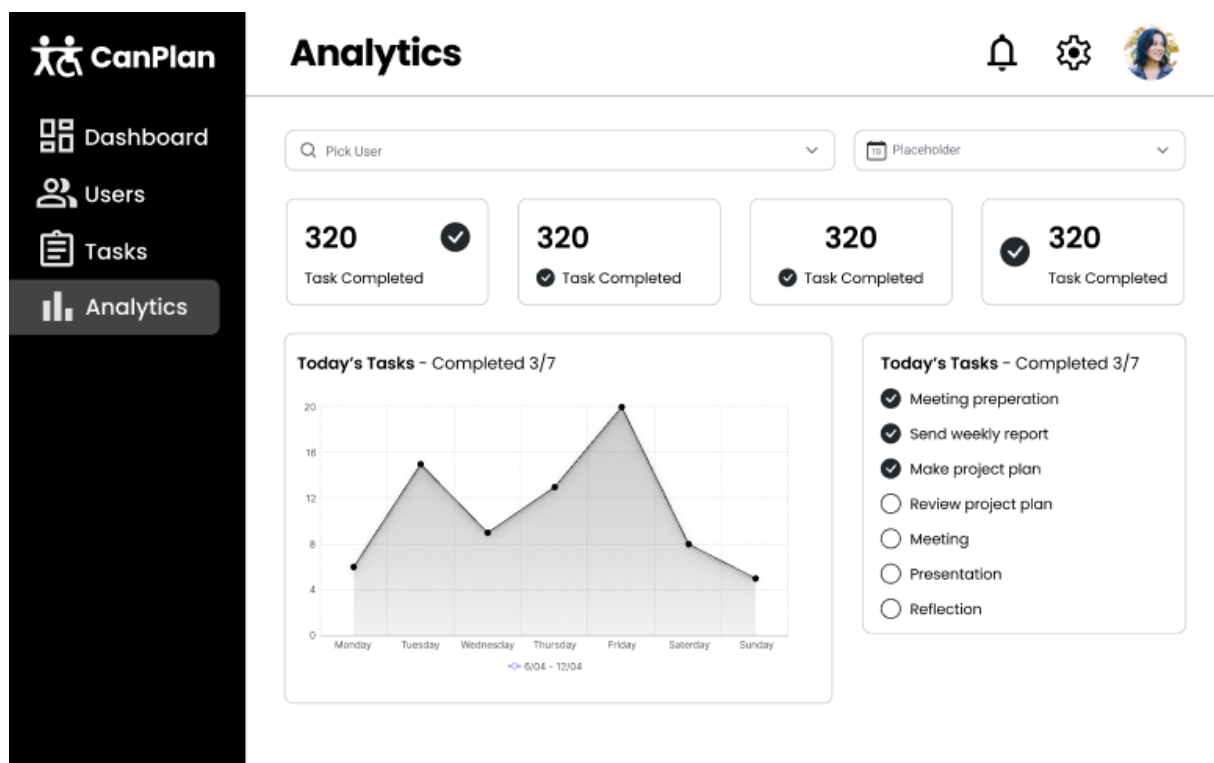
The dashboard design process also explored how an administrative system could work together with the mobile application in a future cloud-based architecture.



Figma Design Admin Dashboard



Figma Design Add Task Screen



Figma Design Analytics Screen

3.5. Implementation of New Features

During the internship, several new features and improvements were implemented within the CanPlan application, including the branching task system, simple mode, reminder functionality, and future cloud expansion possibilities.

BRANCHING TASK SYSTEM

One of the most technically challenging implementations during the internship was the development of a branching task system.

The original version of CanPlan only supported linear task flows, where users completed steps in a fixed sequence. However, certain tasks require multiple possible paths depending on user choices or situations.

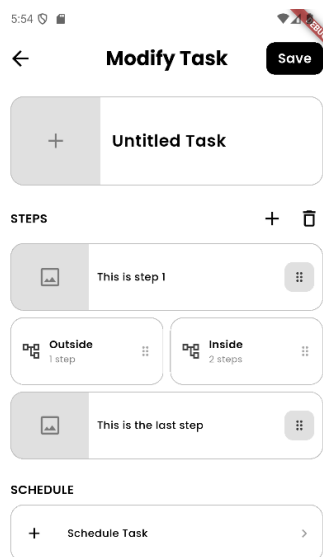
To solve this limitation, a new branching system was designed and implemented.

This system allows:

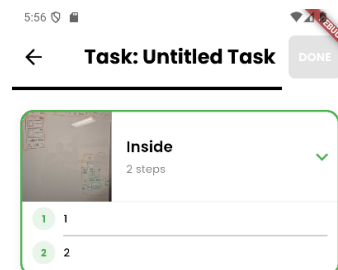
- multiple task flows
- alternative paths
- conditional navigation between steps
- more flexible task execution

During development, a major issue was discovered in the original data structure, which made the initial implementation impossible to complete correctly. Because of this, the entire concept had to be redesigned and rebuilt using a changed data model.

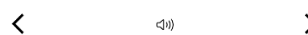
Although this setback required significant additional work, it resulted in a much stronger and more scalable implementation.



New Branching Feature



Choose a path



Branching while doing a task

3.6. User Feedback and Surveys

To better understand user experiences and gather feedback for future improvements, surveys were created during the internship.

Initially, Google Forms was used for this purpose. Later, the system was migrated toward Microsoft Forms because it better aligned with the tools and infrastructure used internally by UVic and CanAssist.

The surveys focused on:

- usability feedback
- accessibility feedback
- feature suggestions
- overall user experience

This feedback helps guide future iterations of the application and supports a more user-centered development process.

3.7. Challenges During Development

During the internship, several technical and organizational challenges were encountered.

One of the largest challenges involved the implementation of the branching system. During development, it became clear that the original data structure was not suitable for supporting complex task flows. As a result, part of the implementation had to be discarded and redesigned from scratch.

Another challenge involved balancing accessibility and functionality. Because the application targets users with different cognitive needs, it was important to simplify the interface without removing important functionality.

In addition, research into cloud expansion introduced several architectural and privacy-related challenges. Decisions regarding backend technologies, hosting providers, and data storage had to take scalability, maintainability, and FIPPA compliance into account.

Finally, several technical difficulties were encountered during the iOS release process. The available Mac Mini did not support the latest required versions of macOS and Xcode, which complicated deployment preparation and release testing.

Despite these challenges, each issue provided valuable learning experiences and contributed to a deeper understanding of software architecture, accessibility, and large-scale application design.

4. Conclusion

During this internship, the CanPlan application was significantly improved both visually and technically. The application received a complete redesign and rebranding focused on accessibility, usability, and consistency.

In addition, several new concepts and features were researched and implemented, including the branching task system, simple mode, reminder functionality, and future cloud expansion possibilities.

The project also explored future-oriented topics such as backend architecture, database technologies, cloud hosting, privacy regulations, and administrative dashboards. This created a strong foundation for future development and scalability of the application.

Although some features remain part of future work, the internship successfully improved the quality, accessibility, and long-term potential of the CanPlan platform.

REFERENCE LIST

- Apple Inc. (n.d.). App sandbox. Apple Developer Documentation. https://developer.apple.com/documentation/security/app_sandbox
- BeeVisual. (n.d.). BeeVisual. <https://www.beevisual.com/>
- Brili. (n.d.). Brili routines and task management for ADHD. <https://brili.com/>
- DigitalOcean. (n.d.). Cloud hosting and VPS solutions. <https://www.digitalocean.com/>
- Flutter. (n.d.). Flutter documentation. <https://docs.flutter.dev/>
- Google. (n.d.). Android security overview. Android Developers. <https://developer.android.com/security>
- Meta Platforms, Inc. (n.d.). React Native documentation. <https://reactnative.dev/>
- MongoDB Inc. (n.d.). MongoDB documentation. <https://www.mongodb.com/docs/>
- Node.js Foundation. (n.d.). Node.js documentation. <https://nodejs.org/en/docs>
- Oracle Corporation. (n.d.). MySQL documentation. <https://dev.mysql.com/doc/>
- PostgreSQL Global Development Group. (n.d.). PostgreSQL documentation. <https://www.postgresql.org/docs/>
- Qcard. (n.d.). Qcard. <http://www.qcard.ca/>
- Render Services, Inc. (n.d.). Render cloud hosting platform. <https://render.com/>
- Routinery. (n.d.). Routinery. <https://www.routinery.app/>
- Spring. (n.d.). Spring Boot documentation. <https://spring.io/projects/spring-boot>
- The Government of British Columbia. (n.d.). Freedom of Information and Protection of Privacy Act (FIPPA). https://www.bclaws.gov.bc.ca/civix/document/id/complete/statreg/96165_00
- University of Victoria. (n.d.). CanAssist. <https://www.canassist.ca/>
- Vercel Inc. (n.d.). Vercel platform documentation. <https://vercel.com/docs>
- Apple Inc. (n.d.). Accessibility features and APIs. Apple Developer Documentation. <https://developer.apple.com/accessibility/>
- Google. (n.d.). Accessibility in Android. Android Developers. <https://developer.android.com/guide/topics/ui/accessibility>
- Interaction Design Foundation. (n.d.). Accessibility: Usability for all. <https://www.interaction-design.org/literature/topics/accessibility>
- Microsoft. (n.d.). Inclusive design. Microsoft Design. <https://inclusive.microsoft.design/>
- Nielsen Norman Group. (n.d.). Accessibility and usability. Nielsen Norman Group. <https://www.nngroup.com/topic/accessibility/>

W3C Web Accessibility Initiative. (n.d.). Introduction to web accessibility. World Wide Web Consortium. <https://www.w3.org/WAI/fundamentals/accessibility-intro/>

World Health Organization. (2023). Disability. <https://www.who.int/news-room/fact-sheets/detail/disability-and-health>